

本文档目的是使用自己的代码来计算 VMC 求基态能量时的梯度数值,而不过于依赖 Netket 中的对应功能函数。以下代码不允许更改:

```
# ===== 环境配置 =====
import jax
import jax.numpy as jnp
import netket as nk
import netket.experimental as nkx
import numpy as np
from pyscf import gto, scf, fci
from flax import nnx
import optax
from functools import partial
from jax import import flatten_util

print(f"JAX version: {jax.__version__}")
print(f"NetKet version: {nk.__version__}")
# ===== 1. H2 分子定义 & FCI 基准 =====
bond_length = 1.4
geometry = [('H', (0., 0., 0.)), ('H', (bond_length, 0., 0.))]
mol = gto.M(atom=geometry, basis='STO-3G', verbose=0)
mf = scf.RHF(mol).run(verbose=0)

# FCI 精确基准
cisolver = fci.FCI(mf)
cisolver.nroots = 4
E_fcis, fcivec = cisolver.kernel()
print("="*60)
print("H2 FCI 基准能量")
print("="*60)
for i, e in enumerate(E_fcis):
    exc = (e - E_fcis[0]) * 27.2114
    print(f"E[{i}] = {e:.8f} Ha | 激发能: {exc:.4f} eV")

# ===== 2. NetKet 哈密顿量和采样器 (仅用于生成样本) =====
ha = nkx.operator.from_pyscf_molecule(mol)

hi = nk.hilbert.SpinOrbitalFermions(
    n_orbitals=2,
    s=1/2,
    n_fermions_per_spin=(1,1),
)

edges = [(0, 1), (2, 3)]
g = nk.graph.Graph(edges=edges)
single_rule = nk.sampler.rules.FermionHopRule(hilbert=hi, graph=g)
sampler = nk.sampler.MetropolisSampler(hi, rule=single_rule, n_chains=16, sweep_size=32)

print(f"Hilbert space size: {hi.size}")

class SingleStateAnsatz(nnx.Module):
    def __init__(self, n_spin_orbitals: int, hidden_dim=16, *, rngs: nnx.Rngs):
        super().__init__()
        self.linear1 = nnx.Linear(n_spin_orbitals, hidden_dim, rngs=rngs, param_dtype=complex)
        self.linear2 = nnx.Linear(hidden_dim, hidden_dim, rngs=rngs, param_dtype=complex)
        self.output = nnx.Linear(hidden_dim, 1, rngs=rngs, param_dtype=complex)

    def __call__(self, x):
        h = nnx.tanh(self.linear1(x.astype(complex)))
        h = nnx.tanh(self.linear2(h))
        out = self.output(h)
        return jnp.squeeze(out)

rngs = nnx.Rngs(21)
model = SingleStateAnsatz(n_spin_orbitals=4, hidden_dim=12, rngs=rngs)
machine, graphdef, params = create_machine(model)
sampler_state = sampler.init_state(machine, params, seed=1)
samples, sampler_state = sampler.sample(machine=machine, parameters=params, state=sampler_state, chain_length=20)
samples.shape # (16,20,4)
```

变分量子蒙特卡洛算法中,基态能量的表达式如下:

$$\langle E(\theta) \rangle = \int E_{\text{loc}}(R, \theta) \pi(R, \theta) dR$$

R 是系统配置, θ 是模型参数 $\pi(R, \theta)$ 是模型在 R 上的密度: $\pi(R, \theta) = \frac{|\psi(R, \theta)|^2}{\int |\psi(R, \theta)|^2 dR}$ $E_{\text{loc}}(R, \theta)$ 是本地能量:

$E_{\text{loc}} = \frac{\hat{H}\psi(R, \theta)}{\psi(R, \theta)}$ 梯度公式推导为: $\frac{d\langle E(\theta) \rangle}{d\theta} = \frac{d}{d\theta} \int E_{\text{loc}}(R, \theta) \pi(R, \theta) dR$ 总之最后的梯度公式为: $\boxed{\nabla_{\theta} E = \mathbb{E}[\pi] \left[E_{\text{loc}} \left(\mathcal{O}[\psi] + \mathcal{O}[\psi^*] \right) \right] - \mathbb{E}[E_{\pi}] \left[E_{\text{loc}} \right] \mathbb{E}[\pi] \left[\mathcal{O}[\psi] + \mathcal{O}[\psi^*] \right]}$ 在代码里, 将变形为: $\boxed{\nabla_{\theta} E = \mathbb{E}[\pi] \left[\left(E_{\text{loc}} - \mathbb{E}[E_{\pi}] \right) \left(\mathcal{O}[\psi] + \mathcal{O}[\psi^*] \right) \right]}$

```
# ===== 5. 纯 JAX 实现的 force-based 梯度计算 =====
@partial(jax.jit, static_argnames=("machine",))
def compute_local_energies(machine, params, sigma):
    """
    计算局部能量  $E_{\text{loc}}(\sigma) = \sum_{\eta} H(\sigma \rightarrow \eta) \psi(\eta) / \psi(\sigma)$ 

    这对应 NetKet 的 local_value_kernel
    """
    eta, H_eta = ha.get_conn_padded(sigma)
    logpsi_sigma = machine(params, sigma)
    logpsi_eta = machine(params, eta)
    logpsi_sigma = jnp.expand_dims(logpsi_sigma, -1)
    return jnp.sum(H_eta * jnp.exp(logpsi_eta - logpsi_sigma), axis=-1)

def statistics(x):
    """计算样本统计量"""
    mean = jnp.mean(x)
    var = jnp.var(x)
    return mean, jnp.sqrt(var / x.shape[0])

@partial(jax.jit, static_argnames=("machine",))
def forces_expect_hermitian(machine, params, sigma):
    """
    ✓ 100% 匹配 NetKet 原版 forces_expect_hermitian
    公式:  $\nabla \langle E \rangle = \langle (E_{\text{loc}} - \langle E \rangle) * \nabla \log \psi^* \rangle$ 
    """
    # 1. 局域能量
    O_loc = compute_local_energies(machine, params, sigma)

    # 2. 能量均值
    O_mean, O_std = statistics(O_loc)

    # 3. 中心化
    O_centered = O_loc - O_mean
    n_samples = sigma.shape[0]
    def log_psi_conj(p, x):
        return jnp.conj(machine(p, x)) # log psi*

    _, vjp_fun = jax.vjp(lambda p: log_psi_conj(p, sigma), params)
    grad = vjp_fun(O_centered / n_samples)[0]

    return O_mean, O_std, grad
```

但是我在尝试对比我基于 JAX 实现的 force-based 梯度计算和 NetKet 原版的 forces_expect_hermitian 时, 发现它们的梯度结果不一致。

```
rngs = nnx.Rngs(21)
model = SingleStateAnsatz(n_spin_orbitals=4, hidden_dim=12, rngs=rngs)
machine, graphdef, initial_params = create_machine(model)

vstate = nk.vqs.MCState(
    sampler=sampler,
    model=model,
    n_samples=1000,
```

```
    seed=1 # 种子放在这里, 替代 sampler.init_state
)
value,grad = vstate.expect_and_forces(ha) #Netket 的梯度计算 和 局域能量估计
#我的实现的梯度计算
energy, energy_std, grad_my = forces_expect_hermitian(machine, initial_params, vstate.samples.reshape(-1,4))
```